ADAPT 2.0

AI-Powered Adaptive Student Learning and Support Platform

Workflow pipeline, flowcharts, test data and measured results

Document date: 11 October 2026. Companion to the ADAPT 2.0 Word documentation.

What is real in this document. Stage 1, the C++17 planning engine, was implemented and tested for this document. All test results, charts and timings come from that engine (10 of 10 planned tests, plus 900 randomised benchmark scenarios) and can be reproduced from the files in the project folder. Stages 2 to 5 (notes library, OCR, AI explanations, quizzes, originality tools, Urdu and low-bandwidth support) are **designed but not built**: they appear in the diagrams and test plans, and no results are reported for them.

Test environment: Ubuntu 24.04, g++ 13.3.0 (-std=c++17 -O2), 2 CPU cores, cloud container. Timings are from this machine, not from a student laptop.

1. End-to-end workflow pipeline

A student uploads notes, ADAPT reads and explains them, the student practises, and weak topics turn into revision tasks that the C++ planner schedules and repairs when real life changes. Blue steps exist and are tested; amber dashed steps are planned.

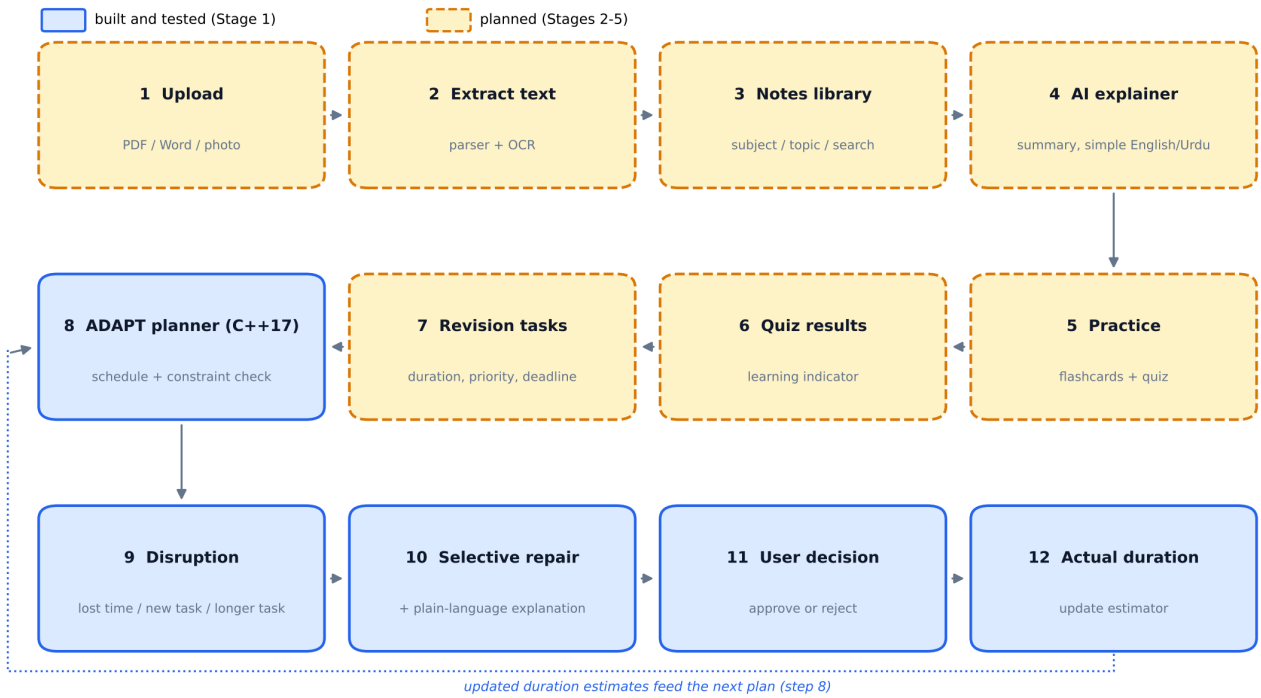


Figure 1. ADAPT 2.0 workflow. The loop from step 12 back to step 8 is how duration estimates improve later plans.

#	Step	What happens	Owner	Status
1	Upload	Student adds a PDF, Word file or photo of notes	Web interface	Planned (Stage 2)

#	Step	What happens	Owner	Status
2	Extract text	Parser or OCR reads the file; unclear regions are flagged, not guessed	Parser + OCR	Planned (Stage 2)
3	Notes library	Notes organised by subject, semester and topic; full-text search	Backend + database	Planned (Stage 2)
4	AI explainer	Summary, key terms and simple-English explanation tied to source passages	AI model	Planned (Stage 3)
5	Practice	Flashcards and a short quiz generated from the notes	AI model	Planned (Stage 3)
6	Quiz results	Scores are treated as a learning indicator, not a measure of understanding	Backend	Planned (Stage 3)
7	Revision tasks	Weak topics become tasks with duration, priority and deadline	Backend	Planned (Stage 3)
8	Plan	Tasks are scheduled around fixed commitments and deadlines	C++ engine	Implemented + tested
9	Disruption	Lost time, a new urgent task or a longer-than-expected task	C++ engine	Implemented + tested
10	Selective repair	Only affected tasks move; every change gets a reason	C++ engine	Implemented + tested
11	User decision	Approve or reject; rejection keeps the old plan and is logged	C++ engine	Implemented + tested
12	Actual duration	Estimator is updated with planned vs actual time	C++ engine	Implemented + tested

2. System architecture

The C++ engine owns scheduling, constraint checking and repair. Language, OCR and similarity tools sit around it and talk to it through the backend, so the core planner keeps working even when an external AI service is down.

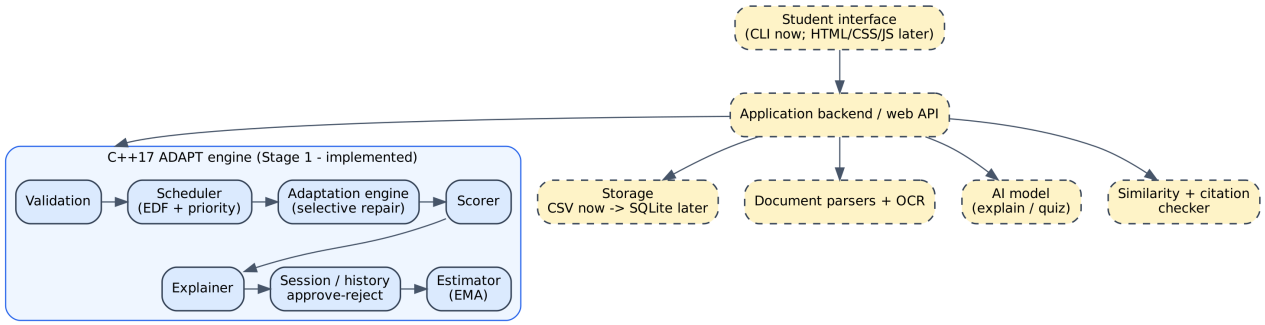


Figure 2. Proposed architecture. Blue: implemented in Stage 1. Amber dashed: planned.

Module	Responsibility	Status
Validation	Rejects malformed tasks; all-or-nothing CSV loading	Implemented
Scheduler	Earliest deadline first, then priority, then shorter task	Implemented
Adaptation engine	Finds affected tasks, builds selective / expanded / full candidates	Implemented
Scorer	Checks hard constraints and computes the cost	Implemented
Explainer	Plain-language reason for every moved or unscheduled task	Implemented
Session and history	Approve / reject, audit log	Implemented
Estimator	Exponential moving average of task durations	Implemented
Storage	CSV now; SQLite later	CSV implemented; SQLite planned
Interface	Command-line now; web interface later	Planned
Notes library + OCR	Upload, extract, organise, search	Planned
AI explainer + quiz	Explanations, flashcards, quizzes	Planned
Originality tools	Similarity check, citation help, advisory AI-writing indicator	Planned

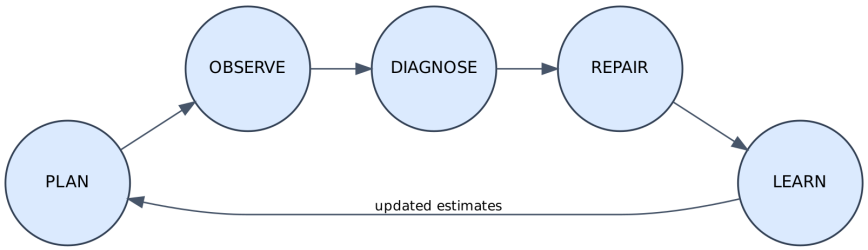


Figure 3. The Plan - Observe - Diagnose - Repair - Learn loop.

3. Repair algorithm flowchart

When a disruption arrives, ADAPT first tries to move only the affected tasks. It widens the search only if the disruption leaves a task unscheduled that was scheduled before. Candidates that break a hard constraint are discarded, and the rest are compared with one cost function.

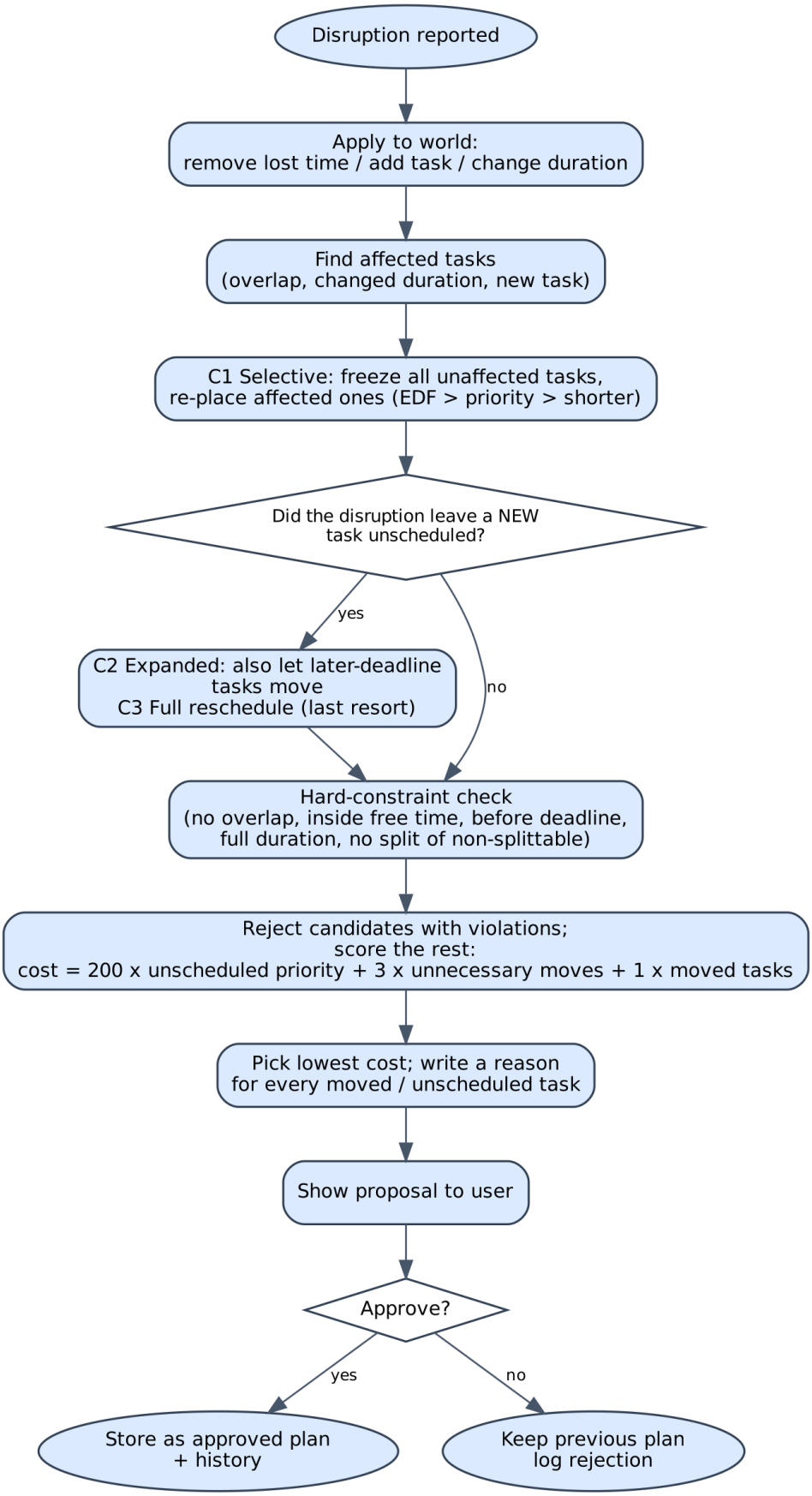


Figure 4. Selective repair with approval. Hard constraints: no overlap, inside free time, before deadline, full duration, non-splittable tasks stay in one piece.

Cost function. Cost = 200 x (priority points of unscheduled tasks) + 3 x (unaffected tasks moved) + 1 x (previously scheduled tasks whose time changed). Deadlines are hard: a task that cannot finish before its deadline is reported as unscheduled, never silently placed late. The weights are design choices; section 7.3 shows how changing them trades completeness against stability.

4. Worked example

Data are the five fictional tasks from the original proposal (Appendix B), with study windows 16:00 to 22:00 on Monday to Thursday and a fixed commitment on Monday 19:00 to 20:00. The disruption removes Monday 16:00 to 18:00.

id	title	duration (min)	priority	deadline	splittable
A01	AI quiz revision	60	5	Mon 18:00	No
A02	OOP practice	60	4	Tue 20:00	Yes
A03	Math revision	45	5	Mon 21:00	Yes
A04	English reading	30	2	Wed 19:00	Yes
A05	Lab report	90	4	Thu 17:00	Yes

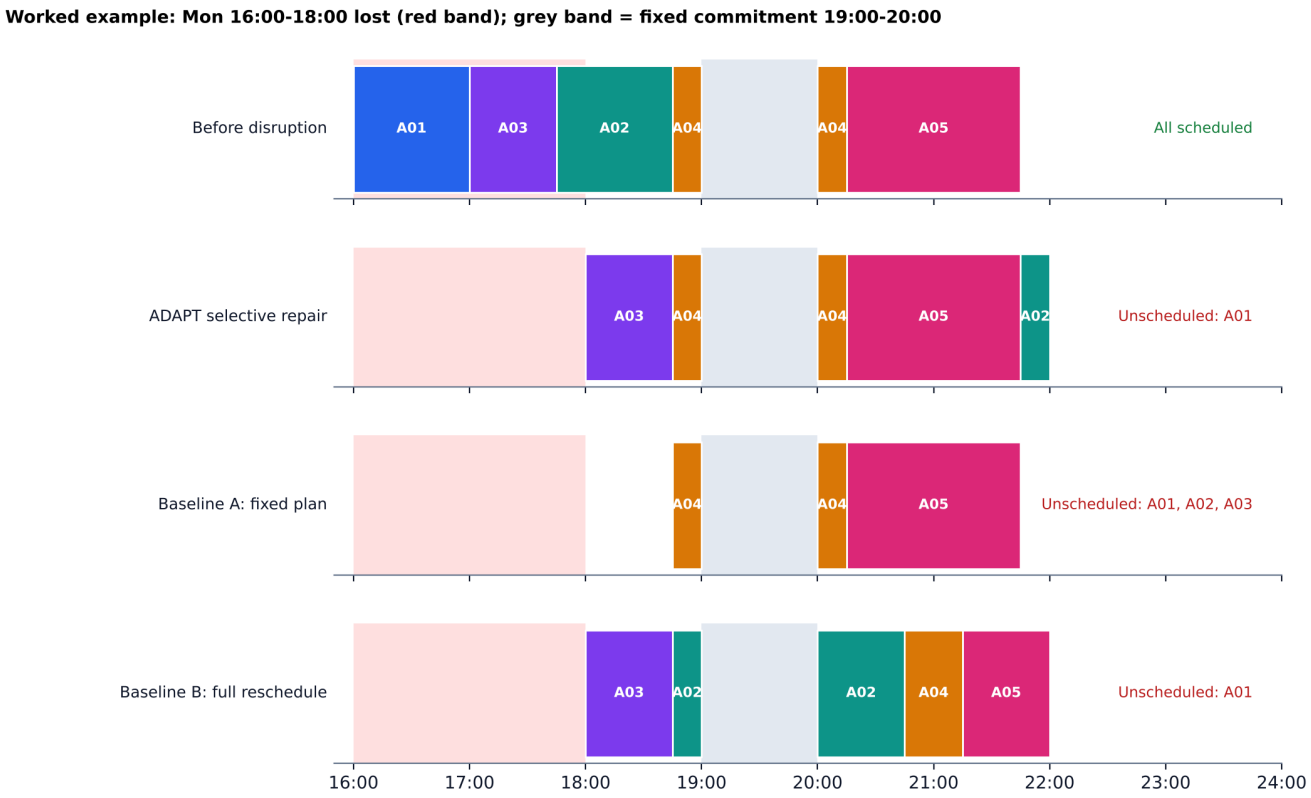


Figure 5. The same disruption handled by ADAPT and the two baselines (output of the real engine).

Method	Scheduled	Unscheduled	Tasks changed	Unaffected tasks moved
A Fixed plan	A04, A05	A01, A02, A03	3	0
B Full reschedule	A02, A03, A04, A05	A01	5	2 (A04, A05)
ADAPT selective repair	A02, A03, A04, A05	A01	3	0

A01 has a Monday 18:00 deadline, so it cannot be rescheduled once 16:00 to 18:00 is gone. ADAPT says so instead of pretending it fits. It leaves A04 and A05 exactly where they were; the full reschedule moves both for no benefit to the student. Engine explanations for this run:

- A01 cannot be scheduled before its deadline (directly affected by the disruption)
- A02 moved to Mon 21:45 (directly affected by the disruption)

- A03 moved to Mon 18:00 (directly affected by the disruption)

5. Roadmap

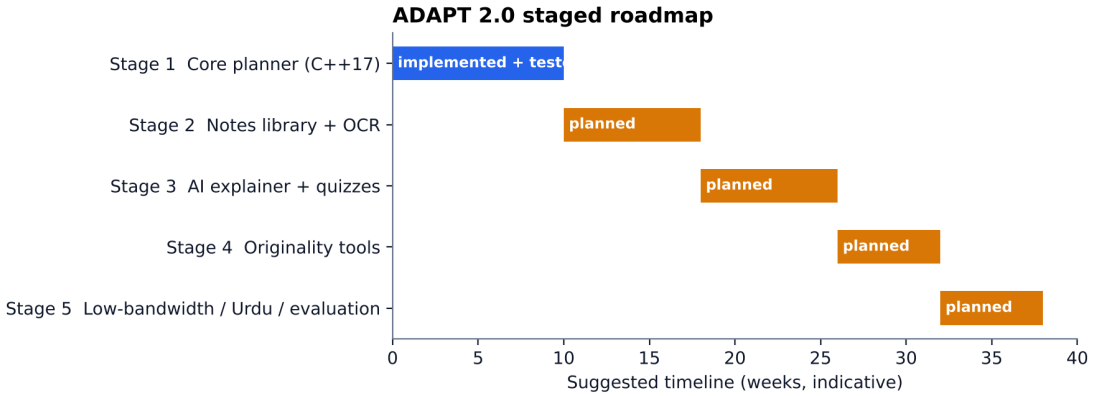


Figure 6. Staged plan. Only Stage 1 is built; week counts are indicative and should be fitted to the course calendar.

6. Test method

- **Unit-style scenario tests (T1 to T10)** from the original proposal's test matrix, run on the demo data above.
- **Randomised benchmark**: 900 scenarios = 3 load levels (8, 14, 20 tasks) x 3 disruption types (lost window, urgent new task, task takes longer) x 100 random seeds (generator seed 2026). 4-day horizon; study windows 09:00-12:00 and 16:00-22:00; about half the days have a 60-minute fixed commitment; task length 30-120 min; priority 1-5; 60% splittable.
- **Disruptions are targeted**: lost windows and longer tasks are applied to a task that is actually scheduled, so every scenario exercises the repair logic.
- **Methods compared on identical inputs**: A fixed plan (affected tasks are dropped, nothing is repaired), B full reschedule (rebuild everything with the same heuristic), ADAPT selective repair.
- **Metrics**: share of scenarios with every task scheduled; unscheduled priority points (sum of priorities of tasks left unscheduled); previously scheduled tasks whose time changed; unaffected tasks moved; hard-constraint violations (checked independently of the planner); runtime.
- **Scale test**: 25 to 400 tasks over 40 days, median of 21 runs per size, against the proposal's target of 100 tasks in under 2 seconds.

7. Results

7.1 Scenario tests T1 to T10

ID	Scenario	Actual output (measured)	Result
T1	No disruption	tasks moved=0, violations=0	PASS
T2	Lost study window Mon 16:00-18:00	affected=3, moved=3, unnecessary=0, unscheduled=1, fixed Mon19-20 used=no, violations=0	PASS
T3	A02 duration 60 -> 120 min	A02 scheduled minutes=120, violations=0, unnecessary moves=0	PASS
T4	Urgent new task N01 (prio 5, 60 min, Tue 12:00)	N01 placed=yes, strategy=expanded, notes=4, violations=0	PASS
T5	Impossible workload (8 extra 2-h tasks due Mon 23:20)	unscheduled=7, false-feasible claim=no, violations=0	PASS
T6	Negative duration, missing title, priority 9	3 validation errors, no crash	PASS
T7	alpha=0.3, E=60, A=80	E: 60 -> 66.0 (expected 66.0); invalid actual ignored=yes	PASS
T8	Second disruption on the approved revised plan	history entries=5, unnecessary moves=0, violations=0	PASS
T9	User rejects suggestion	approved plan unchanged=yes, last log=REJECT prefer to keep Monday evening free""	PASS
T10	Malformed numeric field; duplicate id; valid control file	malformed errors=1, duplicate errors=1, control loaded tasks=2	PASS

10 of 10 passed. These are functional checks on one demo dataset. They show the required behaviours exist; they do not show performance in general.

7.2 Benchmark: all three methods

Method	All tasks scheduled	Tasks scheduled (mean)	Unscheduled priority points	Tasks changed	Unaffected tasks moved	Hard violations
A Fixed plan	0%	12.3 / 14.3	6.85	1.05	0.00	0
B Full reschedule	55%	13.5 / 14.3	2.50	6.75	5.70	0
ADAPT selective repair	55%	13.5 / 14.3	2.48	5.04	4.00	0

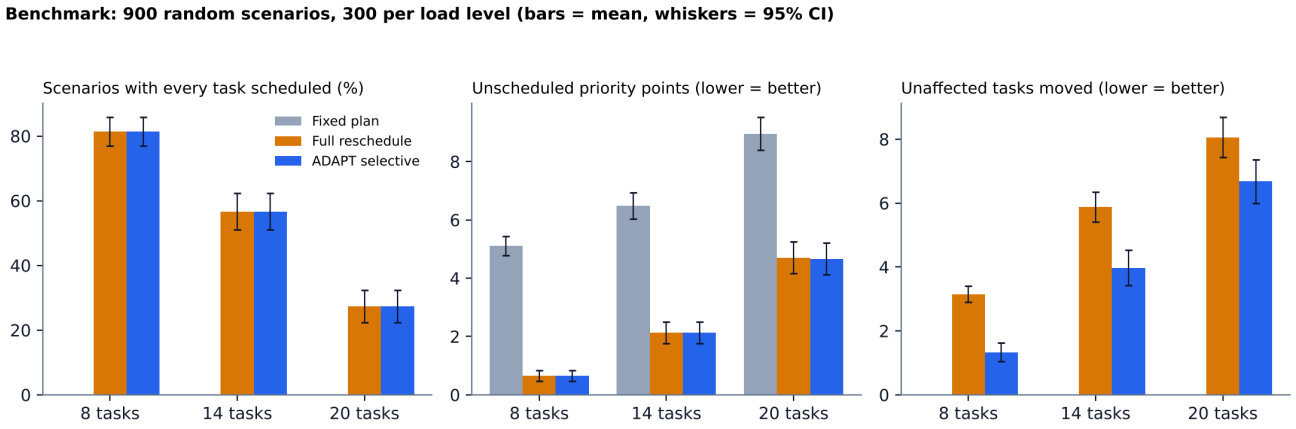


Figure 7. Results by load level (900 scenarios). Whiskers are 95% confidence intervals of the mean.

Load	Method	All scheduled	Unscheduled points	Unaffected moved
8 tasks	A Fixed	0%	5.11	0.00
8 tasks	B Full	81%	0.65	3.15
8 tasks	ADAPT	81%	0.65	1.33
14 tasks	A Fixed	0%	6.49	0.00
14 tasks	B Full	57%	2.13	5.88
14 tasks	ADAPT	57%	2.12	3.97
20 tasks	A Fixed	0%	8.96	0.00
20 tasks	B Full	27%	4.70	8.06
20 tasks	ADAPT	27%	4.66	6.68

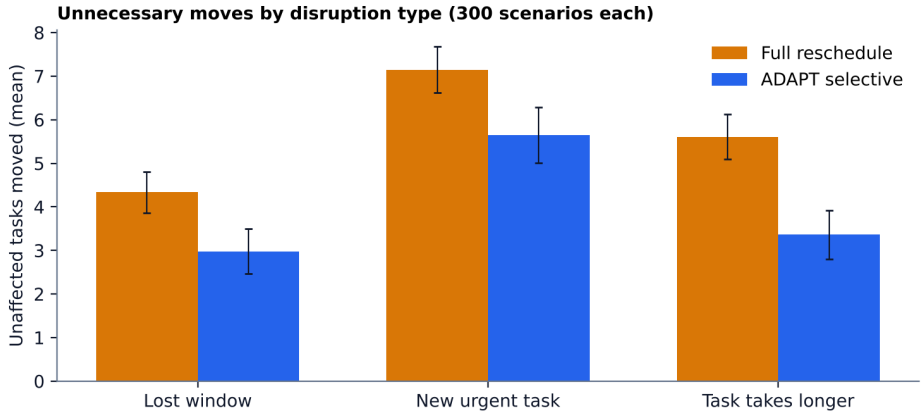


Figure 8. Unaffected tasks moved, by disruption type.

Disruption type	B Full reschedule	ADAPT
Lost study window	4.33	2.98
Urgent new task	7.15	5.65
Task takes longer	5.61	3.36

Main finding. ADAPT completed exactly as many scenarios as the full rescheduler (55%) and left slightly fewer priority points unscheduled (2.48 vs 2.50), while moving 1.70 fewer unaffected tasks per scenario (95% CI +/- 0.18; 5.70 vs 4.00). ADAPT moved fewer unaffected tasks in 47% of scenarios and the same number in 53%; it never moved more. Hard-constraint violations across all 13,500 method runs: 0.

7.3 Choosing the cost weights

The weight on unscheduled priority was swept while everything else was held fixed. Lower weights make ADAPT more conservative: fewer unaffected tasks move, but more work is left unscheduled.

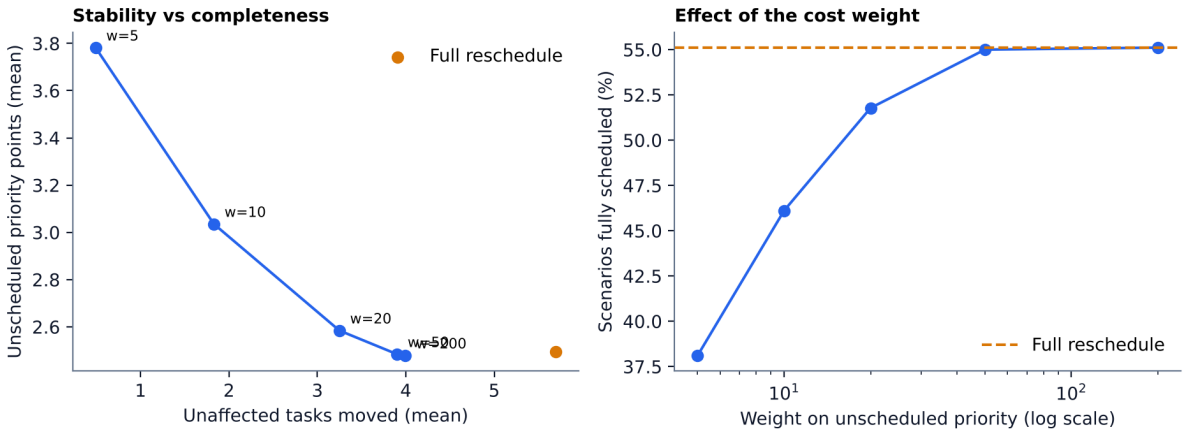


Figure 9. Stability versus completeness as the weight changes.

Weight on unscheduled priority	All scheduled	Unscheduled points	Unaffected moved
200 (default)	55%	2.48	4.00
50	55%	2.48	3.90
20	52%	2.58	3.25
10	46%	3.04	1.83
5	38%	3.78	0.49

The default (200) behaves like the full rescheduler on completeness. A weight near 20 keeps most of the completeness (52% vs 55% fully scheduled) while moving about 19% fewer unaffected tasks than the default. Which point is right depends on how much a student values stability over squeezing in the last task; the choice should be tested with users.

7.4 Runtime

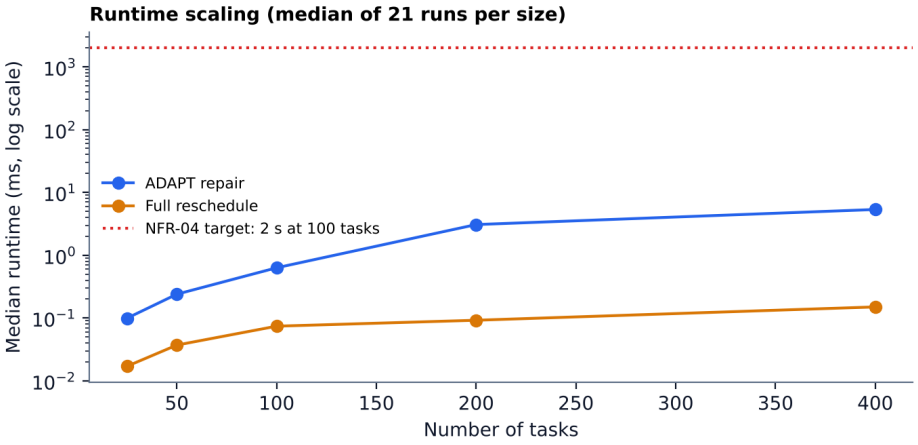


Figure 10. Median repair time versus number of tasks.

Tasks	ADAPT repair, median (ms)	Full reschedule, median (ms)
25	0.10	0.02
50	0.24	0.04
100	0.63	0.07
200	3.07	0.09
400	5.34	0.15

The 100-task target (under 2 seconds) is met by a wide margin: about 0.6 ms. ADAPT is slower than a plain rebuild because it evaluates several candidates, but even at 400 tasks it takes a few milliseconds.

7.5 Raw data sample

First two scenarios of the benchmark (default weights). Full data: results/benchmark.csv (13,500 rows).

scen.	load	disruption	method	strategy	scheduled/total	unscheduled pts	changed	unnecessary	viol.	us
0	8	lost_window	fixed_plan	-	6/8	5	2	0	0	1
0	8	lost_window	full_reschedule	-	8/8	0	5	3	0	3
0	8	lost_window	adapt_selective	selective	8/8	0	2	0	0	33
1	8	lost_window	fixed_plan	-	6/8	7	2	0	0	1
1	8	lost_window	full_reschedule	-	8/8	0	2	0	0	2
1	8	lost_window	adapt_selective	selective	8/8	0	2	0	0	16

8. Defect found by testing

The first benchmark run showed ADAPT moving 4.01 unaffected tasks per scenario, far more than expected. The suspected cause was that ADAPT escalated to wider candidates even when the unscheduled tasks were already unscheduled before the disruption. The trigger was changed to react only to newly unscheduled tasks, but the result barely changed (4.01 to 4.00). The real cause was the cost function: with 200 points per unscheduled priority point and 3 per unnecessary move, ADAPT will reshuffle many tasks to rescue one. That is a modelling choice, not a bug, which is why section 7.3 sweeps the weight. The first run is kept as results/benchmark_v0.1_defect.csv.

9. Limits of these results

- **Synthetic scenarios.** They come from one random generator. Real student timetables may behave differently; the planned user pilot is the real test.
- **ADAPT includes the full reschedule as a fallback candidate.** So it cannot leave more priority points unscheduled than the full reschedule by construction. The meaningful comparison is the number of unaffected tasks moved.
- **Baseline A is deliberately weak.** It drops affected tasks without trying to repair them. Its main use is to show the cost of doing nothing.
- **Timings are from a cloud container**, not a student's laptop.
- **Only Stage 1 is tested.** Nothing here says whether OCR, AI explanations, quizzes or originality tools will work well.

10. Test plans for Stages 2 to 5 (not yet run)

These tables are blank on purpose. Fill the last column only with measured results after each stage is built.

Stage 2 - Notes library and OCR

ID	Scenario	Expected result	Actual result
S2-1	Upload a valid PDF	Text extracted and searchable	not run yet
S2-2	Upload a file above the size limit	Clear rejection message; nothing stored	not run yet
S2-3	Photo of printed text	Character error rate measured against a typed reference	not run yet
S2-4	Photo of handwriting	Unclear regions flagged as low confidence	not run yet
S2-5	Search for a term in three notes	All notes containing the term are returned	not run yet
S2-6	Delete an upload	File, text and search index entries removed	not run yet

Stage 3 - AI explainer and quizzes

ID	Scenario	Expected result	Actual result
S3-1	Explain a notes page	Each claim traceable to a source passage	not run yet

ID	Scenario	Expected result	Actual result
S3-2	Generate 20 quiz questions	Share answerable from the notes, judged by a human reviewer	not run yet
S3-3	AI service unavailable	Planner still works; user told AI features are offline	not run yet
S3-4	Quiz score below 50%	45-minute priority-5 revision task created by the documented rule	not run yet

Stage 4 - Originality tools

ID	Scenario	Expected result	Actual result
S4-1	Copy a paragraph from a source in the corpus	Flagged with a source reference	not run yet
S4-2	Properly quoted and cited paragraph	Not reported as a missing citation	not run yet
S4-3	AI-writing indicator shown	Advisory wording only; no accusation; explains the signals used	not run yet
S4-4	Human-written text by non-native English writers	False-positive rate measured before the feature is shown to anyone	not run yet

Stage 5 - Accessibility and pilot

ID	Scenario	Expected result	Actual result
S5-1	Load the dashboard on a throttled connection	Page weight and load time recorded against a target set first	not run yet
S5-2	Urdu explanation of one topic	Reviewed by a fluent Urdu speaker	not run yet
S5-3	Usability pilot with 5 to 10 students	1 to 5 ratings for clarity, trust and ease of use	not run yet

11. Reproducing these results

All code is C++17 with the standard library only; charts use Python (matplotlib, Graphviz).

```
g++ -std=c++17 -O2 -o run_tests tests/run_tests.cpp && ./run_tests
g++ -std=c++17 -O2 -o bench tests/bench.cpp && ./bench
g++ -std=c++17 -O2 -o demo tests/demo.cpp && ./demo
python3 docs/make_figs.py
```